

PB158 — Practicum in Operating Systems: course orientation

Contents

1 Who runs this course

1.1 What this part shows

1.2 How it was shown

1.3 Why it matters

2 Why this course exists

2.1 What this part shows

2.2 How it was shown

2.3 Why it matters

2.4 Terms

3 What the course covers

3.1 What this part shows

3.2 How it was shown

3.3 The eight topics

3.4 Why it matters

3.5 Terms

4 How you are assessed

4.1 What this part shows

4.2 How it was shown

4.3 Why it matters

4.4 Terms

5 Where to keep learning

5.1 What this part shows

5.2 How it was shown

5.3 Why it matters

6 What must work on your machine

6.1 What this part shows

6.2 Windows

6.3 Linux

6.4 Try it yourself

6.5 Why it matters

6.6 Terms

7 Takeaways

7.1 What this part shows

7.2 How it was shown

7.3 Why it matters

8 Open questions

9 References

10 Appendix – sources from the author's working notes

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Reading this document: *[S<n>]* cites the numbered source list at the end. *[inference]* marks a claim that follows from those sources but is not stated by them. *[speculative]* marks the author's judgement, offered as judgement.

Lecture 1 of PB158, given on 15 September 2026 at FI MUNI. Each section names its slides, starting at slide 1.

1 Who runs this course

1.1 What this part shows

- Slide 5 names the lecturer, Ondřej Krajíček, at `krajicek@mail.muni.cz`, and a co-lecturer and three assistants named in IS MUNI once the course opens.

1.2 How it was shown

- Slide 5 gives two routes: that address, or the IS MUNI contact.

1.3 Why it matters

- Slide 5 sets the direction: material comes from IS MUNI, work goes to FI MUNI GitLab at `gitlab.fi.muni.cz`.

2 Why this course exists

2.1 What this part shows

- Slide 6 states the gap. The cohort can define a system call, a process and virtual memory, and has never attached a tracer.
- Slide 8 separates looking from reciting, and this course grades the first.

2.2 How it was shown

- Slide 7 puts the gap in the curriculum. The 1979 AT&T licence forbade teaching the UNIX source, and courses "went back to theory-only mode or had to use toy simulators" [S5].
- Slide 8 gives the two terms. Monitoring "answers questions you already knew to ask" [V2]. Observability asks a question nobody wrote down in advance [V1], a word from control theory in 1960 [V3].
- Slide 9 asks for a silent explanation of one `f.write()` call. People rate their own understanding lower after an attempt to explain [S1].
- Slide 10: students who only watched a demonstration gained little understanding, and students who predicted first understood measurably better [S2].

2.3 Why it matters

- A written theory test measures recall, and this course measures observation. [inference]
- Lecture 2 proves it, opening with a program whose output disappears when redirected (slide 10).

2.4 Terms

- **practicum** — a course whose assessed output is an observation the student can defend (slide 10).

3 What the course covers

3.1 What this part shows

- One idea carries the eight topics. Every operating-system interface keeps its promise only under conditions nobody wrote down. [inference]
- "The leak is not the abstraction failing; it is the program leaving the scope." [V4] No reasoning names which layer departs from its documentation. Record the boundary crossings and count them [V5].

3.2 How it was shown

- Slide 12 states the idea, and slides 13 to 20 give one topic each. Documentation is a claim, the system call a fact [V21].

3.3 The eight topics

1. **I/O: files and the network** (slide 13). A fast call returns before the work happens [V6]. A socket is a file descriptor on Linux, not on Windows [V7].
2. **The system call boundary** (slide 14). Linux publishes the system call interface and Windows publishes Win32. "Both are stable at the layer they nominate." [V8]
3. **Observing a running system** (slide 15). `strace` and `NtTrace`. A tracer is "an edit to that program" [V22], blind to `io_uring` work in shared memory [V25].
4. **Virtual memory and the page cache** (slide 16). An allocation gives address space. "The page is materialized on first touch, by a fault." [V17]
5. **Protection, shared memory and IPC** (slide 17). Hardware checks every memory access before it completes [V18]. Then pipes, message queues and signals.
6. **Processes, threads and scheduling** (slide 18). A runtime switches its units inside the process, so nothing crosses the boundary [V19]. The empty trace is the finding. [inference]
7. **Post-mortem analysis, with an agent** (slide 19). "A dump is all you get once it is already dead." [V20] An agent adds breadth, not depth [V26].
8. **Leaks in the abstraction** (slide 20). The scope clause is what nobody writes down [V4]. Slide 20 reads the semester's together.

3.4 Why it matters

- Slide 20 sets the pace. No week is fixed to a topic, and three weeks are reviews built from the submissions.

3.5 Terms

- **tracer** — a tool that reports the calls a program makes across a named boundary (slide 15). A profiler and a debugger are different tools.

4 How you are assessed

4.1 What this part shows

- Slide 22 names two components, and a student must pass both.
- **Course credit** (*zápočet* in IS MUNI, slide 22) is earned by submitting at least one of the three assignments. Submission earns it, and a pass is not required.
- **Exam** (*zkouška*, slide 22) is written in a classroom with no AI of any kind.
- The university records the course credit as "requirements fulfilled" or "requirements not fulfilled" [S9].

4.2 How it was shown

- Slide 23 states what an assignment asks for: observe what a system did, and report the conclusion.
- Slide 23 prints the dates, all in 2026. Every deadline is a Friday midnight and every review a Tuesday.
- Assignment 1 (slide 23): set Tue 22 Sep, due Fri 2 Oct, reviewed Tue 6 Oct. Assignment 2: set Tue 6 Oct, due Fri 16 Oct, reviewed Tue 20 Oct. Assignment 3: set Tue 20 Oct, due Fri 30 Oct, reviewed Tue 3 Nov.
- Set to due is ten days, not a fortnight. The deadline precedes its review by three days, so the review uses the submissions (slide 23).
- Slide 24: resubmission is unlimited until the deadline. A fail costs time only.
- Slide 25: the exam has an unaided core plus a removable part. A passed assignment removes its own subject, and the core contents are not decided yet. The submission is the team's, and every member's exam shrinks.

4.3 Why it matters

- The checker returns defects, never remedies. The standard pattern also asks for "concrete revisions" [V15], and slide 24 drops that half.
- A team that iterates to green removes every defect the checker tests for, so the teaching material is the attempt history in Git. [inference]
- "When a measure becomes a target, it ceases to be a good measure" [V16]. Slide 25 chose an unaided core.

- An AI grades both components, with a human on the loop (slide 26). "the action does *not* wait for the human. It proceeds by default" [V10]
- Slide 27 names three failure modes. Detection in Mackworth's task "fell from about 85% at the outset to about 75% within the first 30 minutes" [V24] [V11]. Review becomes "a rubber stamp with an audit trail" [V12]. A critic invents faults on strong work [V14].
- The repair is structural: "the human's job is to adjudicate a raised alarm" [V13], so report a wrong result.
- Slide 28 gives the evidence against the design. Submission limits and regression penalties cut the rate of trial-and-error submission by up to 85% [S6], and PB158 imposes neither. Rapid feedback leads some students to aim at the test cases [S18], and minimal guidance costs a learner with low prior knowledge [S22]. Edwards used the student's own tests instead [S17], and retry-until-mastery teaching is older still [S7].

4.4 Terms

- **human on the loop** — the action proceeds by default. A human catches it afterwards rather than authorising it first [V10].

5 Where to keep learning

5.1 What this part shows

- Slide 29 names four sources, active 13 September 2026.

5.2 How it was shown

- **Julia Evans**, `javns.ca` — tracing, and `strace` inside Docker [S12]. <https://javns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>
- **Raymond Chen**, *The Old New Thing* — Windows history [S14]. <https://devblogs.microsoft.com/oldnewthing/>
- **Brendan Gregg** — flame graphs, eBPF, tracing overhead [S13]. <https://www.brendangregg.com/>
- **LWN.net** — weekly kernel reporting [S15]. <https://lwn.net/>

- **Oxide and Friends** — Cantrill co-created DTrace [S16]. <https://oxide.computer/podcasts/oxide-and-friends>

5.3 Why it matters

- Two carry course material: the overhead figures [S13] and the container diagnosis [S12].

6 What must work on your machine

6.1 What this part shows

- Slide 30 lists the Windows toolchain, slide 31 the Linux one. The Windows side is Microsoft native: WinDbg reads MSVC PDB symbols.

6.2 Windows

- **Visual Studio Community** — tick **Desktop development with C++** (slide 30). <https://visualstudio.microsoft.com/vs/community/>
- **Build Tools** is the wrong package, because it needs a licence (slide 30).
- **VS Code** and the C/C++ extension, on both platforms (slide 30). <https://code.visualstudio.com/>
- **Sysinternals Suite** <https://learn.microsoft.com/sysinternals/>
- **NtTrace** — the `strace` counterpart. <https://github.com/rogerorr/NtTrace>
- **WinDbg** — crash dumps. <https://learn.microsoft.com/windows-hardware/drivers/debugger/>
- **WSL2** — the route to the Linux side (slide 30). <https://learn.microsoft.com/windows/wsl/install>
- **Python** and **uv** — "An extremely fast Python package and project manager, written in Rust" [S21]. <https://docs.astral.sh/uv/>

6.3 Linux

- **VS Code** <https://code.visualstudio.com/>

- **gcc**, **make**, **libc headers**, and **strace** and **gdb** — the distribution's packages (slide 31).
- **Python** and **uv** [S21]. <https://docs.astral.sh/uv/>
- The course **dev container** carries the whole Linux side [S10].
- A **ptrace**-based tracer in a container needs the **SYS_PTRACE** capability and a relaxed seccomp profile [S11].

6.4 Try it yourself

- On Linux, first run (slide 31): `gcc --version`, `strace --version`, `gdb --version`, `uv --version`. Then run `strace -f -e trace=write python3 -c "print('hi')"`. Look for a `write` line.
- On Windows, open the **Developer Command Prompt for VS** first. Run `cl` there. Then run `wsl --status`. An ordinary PowerShell reports "not recognised", because one installation "can set twenty or more environment variables" [S20].

6.5 Why it matters

- Visual Studio Community needs 20 to 50 GB [S19]. Install it before the first lab.
- Slide 32 names both traps, and slide 31 records the faculty machines.

6.6 Terms

- **dev container** — a running container with a defined tool stack, described by a file committed with the code [S10].

7 Takeaways

7.1 What this part shows

- **[T1]** The course grades looking, not reciting (slide 3).
- **[T2]** The course works downward, from the code the student writes to the operating system under it (slide 3).

7.2 How it was shown

- Slide 8 marks [T1], slide 13 marks [T2], and slide 33 restates both.

7.3 Why it matters

- Slide 33 closes on three consequences. The course never grades recall. A doubtful attempt is worth submitting, because the review lectures are built from failures. The grading arrangement has named failure modes, so report a wrong result [V11] [V12] [V13].

8 Open questions

Undecided on 15 September 2026, from slides 22, 25 and 26.

- **How much AI assistance the homework permits.** Slide 22 states a rule for the exam only.
- **Resits.** University regulations govern the number of attempts [S9].
- **Anonymity in the review lectures.** With few teams, a mistake is attributable on sight. [speculative]

9 References

- [S1] Rozenblit, L. & Keil, F. (2002). *The misunderstood limits of folk science: an illusion of explanatory depth*. Cognitive Science 26, 521–562. <https://cogdevlab.yale.edu/sites/default/files/files/rozenblit%20&%20keil%20%202002.pdf>
- [S2] Crouch, C., Fagen, A., Callan, J. P. & Mazur, E. (2004). *Classroom demonstrations: learning tools or entertainment?* American Journal of Physics 72, 835. <https://works.swarthmore.edu/fac-physics/203/>
- [S3] Neville-Neil, G. V. et al. (2017). *Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation*. USENIX ;login:. https://www.usenix.org/system/files/login/articles/login_spring17_03_neville-neil.pdf
- [S4] Tian et al. (2025). arXiv:2506.07955 — on teachers working with large-language-model grading in classrooms. Recorded title contradicted on re-reading, and no exact title confirmed in this build. <https://arxiv.org/abs/2506.07955>

- **[S5]** Tanenbaum, A. S. (2016). *Lessons Learned from 30 Years of MINIX*. Communications of the ACM. <https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf>
- **[S6]** Lawrence, Foss & Urazova (2023). On submission limits and regression penalties in automated assessment. *ACM Transactions on Computing Education*. <https://dl.acm.org/doi/10.1145/3591210>
- **[S7]** Eyre, H. L. (2007). *The Keller Plan and the Personalized System of Instruction*. *The Behavior Analyst Today* 8(3). <https://files.eric.ed.gov/fulltext/EJ800986.pdf>
- **[S8]** Bainbridge, L. (1983). *Ironies of Automation*. *Automatica* 19(6). <https://gwern.net/doc/sociology/technology/1983-bainbridge.pdf>
- **[S9]** Masaryk University. *MU Study and Examination Regulations*, current edition as published on the official notice board. The page carries the edition in force together with its later modifications. <https://www.muni.cz/en/about-us/official-notice-board/mu-study-and-examination-regulations>
- **[S10]** *Development Containers*. containers.dev. <https://containers.dev/>
- **[S11]** *Create a Dev Container*. Visual Studio Code documentation. <https://code.visualstudio.com/docs/devcontainers/create-dev-container>
- **[S12]** Evans, J. (2020). *Why strace doesn't work in Docker*. jvns.ca. <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>
- **[S13]** Gregg, B. *brendangregg.com*. <https://www.brendangregg.com/>
- **[S14]** Chen, R. *The Old New Thing*. Microsoft Developer Blogs. <https://devblogs.microsoft.com/oldnewthing/>
- **[S15]** *LWN.net*. <https://lwn.net/>
- **[S16]** Cantrill, B. & Leventhal, A. *Oxide and Friends*. <https://oxidecomputer/podcasts/oxide-and-friends>
- **[S17]** Edwards, S. H. (Virginia Tech). Web-CAT project paper, `pos27-Edwards.pdf`. Exact title unconfirmed in this build. <https://web-cat.org/publications/pos27-Edwards.pdf>
- **[S18]** Luxton-Reilly, A., Tempero, E. & Arachchilage, N. ACE '23, Australasian Computing Education Conference. Exact title unconfirmed in this build. <https://valerio-terragni.github.io/assets/pdf/luxton-ace-2023.pdf>
- **[S19]** Microsoft Learn. *Visual Studio 2026 system requirements*. <https://learn.microsoft.com/en-us/visualstudio/releases/2026/vs-system-requirements>
- **[S20]** Microsoft Learn. *Use the MSVC toolset from the command line*. <https://learn.microsoft.com/en-us/cpp/build/building-on-the-command-line>
- **[S21]** Astral. *uv — an extremely fast Python package and project manager*. <https://docs.astral.sh/uv/>

- **[S22]** Kirschner, P., Sweller, J. & Clark, R. (2006). *Why Minimal Guidance During Instruction Does Not Work*. Educational Psychologist 41(2). https://itgs.ict.usc.edu/papers/Constructivism_KirschnerEtAl_EP_06.pdf
- **[V1]**, **[V2]**, **[V3]** *Observability vs Monitoring* — author's working notes.
- **[V4]**, **[V5]**, **[V6]** *Where the Abstraction Leaks* — author's working notes.
- **[V7]** *Sockets and Network I/O* — author's working notes.
- **[V8]** *Windows and Linux place the stable syscall boundary at different layers* — author's working notes.
- **[V9]**, **[V23]** *Observing a Running System* — author's working notes.
- **[V10]**, **[V11]**, **[V12]**, **[V13]**, **[V24]** *Human on the Loop* — author's working notes.
- **[V14]**, **[V15]** *Generative Actor Critic as an Agentic Orchestration Pattern* — author's working notes.
- **[V16]** *Goodhart's Law* — author's working notes.
- **[V17]** *Virtual Memory and the Page Cache* — author's working notes.
- **[V18]** *Memory Protection* — author's working notes.
- **[V19]** *Processes, Threads and Scheduling* — author's working notes.
- **[V20]** *Crash Dump Analysis on Windows and Linux* — author's working notes.
- **[V21]**, **[V26]**, **[V27]** *AI Agents as Debugging Operators* — author's working notes.
- **[V22]**, **[V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot* — author's working notes.

10 Appendix — sources from the author's working notes

The talk cites material from a private research vault. Each entry below summarises one such note and lists every external source it draws on.

- appendix/V1.md
- appendix/V2.md
- appendix/V3.md
- appendix/V4.md
- appendix/V5.md
- appendix/V6.md
- appendix/V7.md
- appendix/V8.md
- appendix/V9.md

- [appendix/V10.md](#)
- [appendix/V11.md](#)
- [appendix/V12.md](#)
- [appendix/V13.md](#)
- [appendix/V14.md](#)
- [appendix/V15.md](#)
- [appendix/V16.md](#)
- [appendix/V17.md](#)
- [appendix/V18.md](#)
- [appendix/V19.md](#)
- [appendix/V20.md](#)
- [appendix/V21.md](#)
- [appendix/V22.md](#)
- [appendix/V23.md](#)
- [appendix/V24.md](#)
- [appendix/V25.md](#)
- [appendix/V26.md](#)
- [appendix/V27.md](#)